Comparability and Reproducibility in HPC Applications' Energy Consumption Characterization

Tom Hilgers
tom.hilgers@rwth-aachen.de
RWTH Aachen University
IT Center
Aachen, Germany

Radita Liem
liem@itc.rwth-aachen.de
RWTH Aachen University
IT Center
Aachen, Germany

ABSTRACT

The computational power of HPC systems continues to grow, and improving their energy efficiency is a critical issue for the field in the face of climate change and energy crises. One major aspect of energy optimization lies in the applications run on the systems themselves. In this work, we are looking into comparing energy consumption between different systems using a characterization process based on the recent energy characterization paper as a reference and starting point for other data centers to assess their application's energy patterns. We demonstrated that we could use the methods from the starting paper, replicate the findings, and extend the work to more applications and more systems. Our work acts as a proof of concept for a repository of HPC applications' energy patterns in our future work.

CCS CONCEPTS

• **General and reference** → **Measurement**; • **Hardware** → **Energy metering**.

KEYWORDS

Energy Consumption Characterization, Energy Analysis, HPC

ACM Reference Format:
Tom Hilgers and Radita Liem. 2024. Comparability and Reproducibility in HPC Applications' Energy Consumption Characterization. In *The 15th ACM International Conference on Future and Sustainable Energy Systems (E-Energy '24), June 4–7, 2024, Singapore, Singapore.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3632775.3662162

1 INTRODUCTION

The computational power of High Performance Computing (HPC) systems continues to increase, defying the so called death of Moore's Law [31, 33]. In this pursuit, the energy consumption of new products has risen in recent years [9, 32]. For instance, a contemporary high-end Central Processing Unit (CPU) like the Intel Xeon Platinum 8468 has a Thermal Design Power (TDP) of 350 W [15], a significant increase from 10 years ago when CPUs, such as the Intel Xeon E5-2697 v2, had a TDP of 130 W [14]. This substantial surge in energy consumption is particularly evident in large computing clusters housing hundreds or thousands of these units.

These increasing power demands of HPC hardware stand in stark contrast to efforts to reduce energy consumption caused by climate change concerns and high energy prices all around the globe. HPC systems must balance these factors with the increasing demand for computational power while operating within limited power and monetary budgets. The heart of the energy consumption in an HPC system comes from the application itself. This causes various efforts to do a fine-grained optimization of the applications to improve the energy consumption [22]. Therefore, we see that analyzing and characterizing the energy consumption patterns of the HPC applications running on these systems is a critical first step for improving the energy consumption. Energy characterization data, like timeline or total consumption charts, can help improve energy efficiency efforts, provide references for cross-checking the system and applications' configurations, and determine other influencing factors. It is evident that various stakeholders can therefore benefit, such as application developers interested in optimizing consumption, or hardware procurers intent on making more informed decisions about hardware purchases based on application energy profiles.

Currently, there are numerous approaches, methods, and software tools to perform such analyses in existing literature [6, 13, 34], which can be difficult to choose from a methodological point of view or with regard to the use of performance characterization tools. With these abundant choices, we decided to use one of the latest works in energy characterization from 2022 by Kraljic et al. [18] as a starting point for a comparison study and developing a standardized energy characterization process. Modern HPC systems usually consist of CPU and Graphic Processing Unit (GPU) systems, so we need to ensure that the methodology we use can work with both. The work utilizes the Like I Knew What I Was Doing (LIKWID) framework, which has recently established itself as a popular and easy-to-use tool that enables the collection of energy data and makes it relevant for modern HPC systems. We use their methodology to assess the impact of frequency changes on the energy patterns and data deviation in our HPC systems using three applications: STREAM [23], GROMACS [1], and CIAO [10] that represent the typical workloads in HPC systems.

We managed to demonstrate that we can use the methods offered by Kraljic et al. [18] that were originally used for GPU in our CPU cluster in the same fashion and get a comparable performance trend. The methods can also be extended into several other HPC applications that are not listed in the original work and achieve comparable performance results with other papers that also perform energy

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
E-Energy '24, June 4–7, 2024, Singapore, Singapore
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0480-2/24/06
https://doi.org/10.1145/3632775.3662162

measurements in the case of STREAM and GROMACS. This shows that the methods are generalizable and can be extended to more systems as we have demonstrated by performing experiments in two different HPC systems: CLAIX-18 and CLAIX-23. Consequently, our findings are a foundation for further investigations, by providing a generalized investigation practice and initial data to compare and build upon. As a next step, we intend to expand our work to build a repository of energy data, as it has been done by open science initiatives and in other HPC fields [24], that we will discuss further in the future works.

This paper is structured as follows. Section 2 provides an overview of current methods and recent work in investigating energy consumption and application characterization. Section 3 outlines the methodology and process we use for our characterization. In Section 4, the results of our measurements are presented, highlighting noteworthy characteristics. These findings are then discussed and analyzed in Section 5, after which Section 6 concludes by summarizing the key insights and presenting future research directions.

2 BACKGROUND AND RELATED WORKS

2.1 Energy Measurement Methods

There are various methods for analyzing the energy consumption of HPC applications. The most relevant of these are summarized, analyzed, and evaluated in the works of Benedict [6], Ilsche [13], Younge et al. [34]. In this context, the methods can be categorized as either out-of-band or in-band. These have different advantages and disadvantages, as well as distinct spatial resolutions.

Out-of-band measurements are carried out with dedicated external measuring devices that are connected to a whole computing system or piece of hardware [5]. Such measurements have the advantage that they do not generate any overhead and that the devices are often calibrated for high accuracy. However, the setup and installation of such devices are often expensive, complicated, and limited to the node or component levels. In-band measurements are not conducted with the help of external measuring devices but instead using integrated software-accessible sensors, which enable a high spatial resolution. These in-band measurements are often preferred due to their ease of use, which suits our needs to assess multiple applications in high spatial resolution. In the case of CPUs, the sensors are exposed to the user via hardware counters, i.e. Model-Specific Registers (MSRs). GPUs require separately installed drivers and interfaces. NVIDIA exposes its sensors via interfaces such as nvidia-smi [26] for static and NVML [27] for dynamic access. Similarly, AMD provides rocm-smi [2] for static and rocprof [3] for dynamic access. These different access methods are combined in energy profiling tools or frameworks that simplify the process and have a wide compatability. Our work focuses on the detailed view of the application's energy consumption; therefore, we are using in-band measurements with LIKWID.

LIKWID is one of the popular in-band energy profilers for HPC systems that is often used as a base of other energy measurement tools [8, 11]. It enables a detailed analysis of the system architecture, computing power, energy consumption, and cache utilization. It supports a wide range of (co-)processor architectures and is suitable for optimizing parallel applications. Detailed performance data is always helpful for developers and system administrators to identify bottlenecks and improve the efficiency of applications on HPC systems. LIKWID has the timeline and wrapper modes that enable data collection from specific applications with a given measurement interval and without the need for instrumented code.

2.2 Energy Consumption of HPC Applications

Investigating energy consumption in HPC often focuses on individual aspects such as specific hardware architectures, clusters, and data centers with their own choice of certain energy modeling. These investigations are conducted on applications commonly used in the HPC environment, such as the work from Hackenberg et al. [12] that uses the SPEC MPI benchmark and focuses on the impact of load balance.

Jiao et al. [16] characterize the energy consumption and energy efficiency of GPUs based on several HPC applications. At the time of publication, GPUs and accelerators were still a less integral part of HPC systems [17] and, therefore, still novel in the field. This study works quite similarly to our main reference paper for the methodology from Kraljic et al. [18]. In the work of Kraljic et al. [18], they expanded the LIKWID feature set to AMD GPU platforms and presented a methodology for investigating the effect of frequency scaling on the energy consumption of HPC applications. The work of Khaleghzadeh [17] investigates the energy consumption behavior of the hardware at different core and memory frequencies. They utilize out-of-band measurements, as in-band measurements were still in their infancy at the time of the investigation. Calore et al. [7] study the energy efficiency of a novel ARM based CPU architecture. Specifically, they investigate its suitability for HPC clusters using a benchmark and two HPC applications.

Our work focuses on the characterization that should be independent of the architecture, which is similar to the work done by Song et al. [30] where they are using the HPC Challenge benchmark. However, we are also assessing the comparability of our results with other published papers because we want to use our work as a proof of concept for building a repository of applications' energy patterns from different systems and data centers. Additionally, to enable more studies in the future, we are also looking into the characterization process to find aspects that can be improved.

3 METHODOLOGY

As a starting point of our work, we start by replicating and expanding the work of Kraljic et al. [18] with several new applications and different hardware. By doing this we

- test their method's transfer- and applicability to other hard- and software environments.
- create a proof of concept for a comparable and standardized energy characterization method.
- solidify the process of characterizing energy consumption of common HPC applications.

The methodology we chose is already applied for GPUs, and we want to test it to CPUs since modern HPC systems mostly consist of the combination of CPUs and GPUs. We want to ensure that the energy characterization method and the tools can work in both systems or even in hybrid settings. Our work aims to provide an example of how the method can be replicable across different hardware and applications. The community interested in energy

characterization can also use our exercises for further replication and as a base for comparing energy patterns.

3.1 Measurement Interval Deviation Analysis

LIKWIDs performance counter measurement interval is freely adjustable, resulting in a varying strength of overhead influence. While LIKWID only generates a small overhead [29], other factors specific to our system configuration might influence the results. We therefore replicate the investigation of the minimum required interval length of Kraljic et al. [18] to find an optimal value. The underlying application that we use for this is STREAM. We examine intervals ranging from 10 ms to 250 ms. Each measurement has a period timestamp, which we compare with the mathematically calculated overhead-free timestamp. The percentage difference between the timestamps is calculated using

$$deviation[i] = \left(\frac{t_{measured}[i]}{i \cdot I} - 1 \right) \cdot 100$$

for $i \in [1, n_periods]$, where $t_{measured}$ is the timestamp in the `Total Runtime [s]` field in the LIKWID measurements, I is the interval, and $n_periods$ the total number of measurements done by LIKWID.

3.2 Frequency Scaling

We also investigate the effect of CPU frequency scaling on energy consumption. The frequencies range from 1 GHz to 3.5 GHz in steps of 0.5 GHz as well as additionally the maximum turbo frequency of 3.7 GHz. This investigation is performed for all three applications using all available cores.

During pre-testing, we found that LIKWID often reports impossible values in its `Power(W)` output field, far exceeding the TDP of the system. However, it also reports the total energy consumed since the previous measurement in Joule in the `Energy(J)` output field. We can convert these values to Watt using

$$power[i] = \frac{energy[i]}{t[i] - t[i-1]}$$

for $i \in [1, n_periods]$, where $t[i]$ is the respective timestamp. These converted values were much closer to what is expected, rarely exceeding the TDP, despite both output fields reading the same register. To illustrate the deviation between the two values, we include them in our results section. Furthermore, we always use the `Energy(J)` field for energy-related results.

3.3 Applications

We use three commonly used applications in HPC to verify the methodology and evaluate its transferability.

GROMACS [1] is a software designed to simulate molecules, proteins, lipids, and other biomolecules in chemistry, biology, and related disciplines. The software is widely used and was already subject to energy measurements [19, 28]. Using GROMACS will enable us to cross-check results and expand the existing library of energy measurements.

CIAO [10] is a Navier-Stokes solver for turbulent reacting flows in complex geometries and is used in simulations of internal combustion engines. This solver is extensively used at RWTH Aachen University. Therefore, a detailed energy characterization is relevant to our daily use case.

Table 1: The systems under investigation.

Hardware	CLAIX-18-MPI	CLAIX-23-HPC
CPU	2x Intel Xeon Platinum 8160	2x Intel Xeon 8468
Cores	24 per CPU	48 per CPU
Base Frequency	2.1 GHz	2.1 GHz
Turbo Frequency	3.7 GHz	3.8 GHz
Last Level Cache	33 MB per CPU	105 MB per CPU
TDP	150 W per CPU	350 W per CPU
Main Memory	192 GB	256 GB

Table 2: Application configuration.

Application	Version	Dataset	Configuration
STREAM	5.10	Default	264 MB & 840 MB
GROMACS	2021.5	BenchMEM [20]	10000 steps
CIAO	2022.3	Nista et al. [25]	Liem et al. [21]

In addition to these two real-world applications, we also investigate the widely used STREAM memory-bandwidth microbenchmark [23] to further validate and replicate the measurement methods outlined in the reference work.

3.4 Test Systems

The systems under investigation and the configuration of the applications can be found in Tables 1 and 2. To test the transferability of the proposed method we perform our measurements on CPUs. The majority of our research is focused on our CLAIX-18 cluster. We include additional results for the measurement interval deviation collected with beta-access on our new CLAIX-23 cluster, that add to the investigation of transferability. The data collection is performed with *likwid-perfctr* version 5.2 on CLAIX-18, version 5.3 on CLAIX-23, and the ENERGY event set. We omit the first measurement iteration in all of our analyses because we found that these report implausible values. The compilations are done with the Intel compiler and IntelMPI, version 2021.6.0 on CLAIX-18 and version 2023.1.0 on CLAIX-23. Cache effects in STREAM are prevented by setting large array sizes.

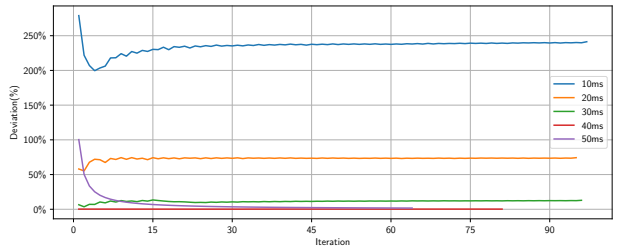
The frequency setting is done via a SLURM directive, with which we set the minimally and maximally allowed frequency to the same value combined with the performance power governor. This has proven to be the most suitable and safest method in our cluster because it automatically resets to default values after a batch job has finished.

4 MEASUREMENT RESULTS

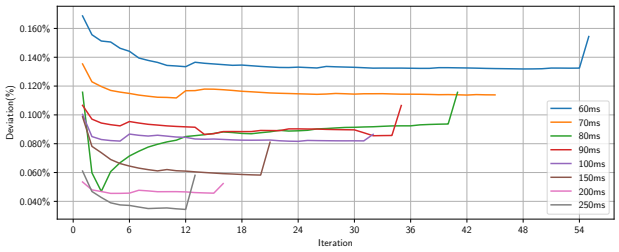
This section presents the data collected during the applications' execution and highlights noteworthy characteristics. We first present the results of the measurement interval investigation. This is followed by the frequency scaling results grouped by application.

4.1 Measurement Interval Deviation

4.1.1 CLAIX-18. As Figure 1 shows, longer intervals reduce the observed deviation from the expected values. This deviation peaks in the initial iterations, triggered by overheads in the initialization

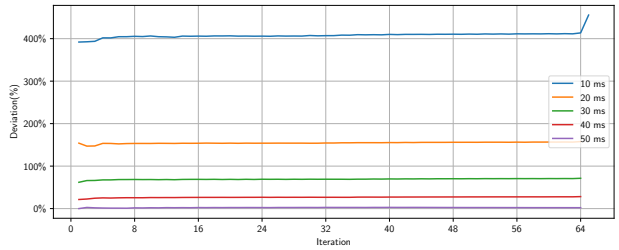


(a) Short measurement intervals (≤50 ms)

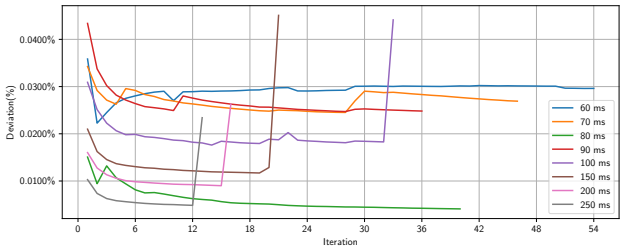


(b) Long measurement intervals (>50 ms)

Figure 1: The deviation for different measurement intervals on CLAIX-18.



(a) Short measurement intervals (≤50 ms)



(b) Long measurement intervals (>50 ms)

Figure 2: The deviation for different measurement intervals on CLAIX-23.

of the measurement and software. In subsequent iterations, the deviation then converges and only minorly fluctuates. The last iteration also displays a relatively higher deviation for intervals longer than 50 ms.

The behavior of the intervals is generally similar, with few outliers. For example, the 50 ms interval exhibits a generally worse deviation than the shorter 40 ms interval, even exceeding 30 ms and 20 ms at times. Although the deviation of the 50 ms interval decreases rapidly, it converges to a higher value than that of 40 ms. Another outlier is 80 ms, which exhibits a steep drop in deviation over the first six iterations, reaching that of 200 ms. Subsequently, it converges upwards, behaving inversely to all other long intervals.

All intervals of 60 ms or longer demonstrate a low deviation, indicating only a minor influence of overheads. These deviations range from approximately 0.036% to 0.165%. To keep our configuration as close as possible to the reference paper, we conduct all frequency scaling measurements with an interval of 100 ms.

4.1.2 CLAIX-23. Figure 2 shows that the behavior on this system is similar to CLAIX-18. Longer intervals also consistently reduce the observed deviation. However, there are noteworthy differences in the exact values, as the short measurement intervals show a higher deviation that exceeds 400% instead of the previous 250%. For the longer intervals, the values range between below 0.01% and 0.04%, much lower than on the other system. These long intervals also exhibit a more mixed behavior in that some have a deviation that is lower, like 80ms, or higher, like 90ms, than expected. We can again see spiking in the last iteration, but on this system, it is only for some intervals. These results underline that the 100 ms interval is a suitable configuration across systems.

4.2 Frequency Scaling

In this subsection, we present the results of the frequency scaling, which were exclusively collected on the CLAIX-18 system. We begin with the results obtained from STREAM, followed by GROMACS, and then CIAO. We provide tables of the deviation and total consumption results to complement the graphs.

4.2.1 STREAM. As shown in Figure 3a and Table 3, there are significant differences between the power and converted energy measurements, ranging between 20% and 73%, which roughly trend downwards with increasing frequency. Figure 3b shows that frequencies set above 2.5 GHz no longer affect the actually reached frequency, which remains around 2.75 GHz. Furthermore, the general deviation and occasional fluctuations from the set values also increase with increased frequencies. As shown in Figure 3c and Table 4, the lowest frequency consumes the most energy, followed by a local minimum at 1.5 GHz, and an increase to a maximum at 2.5 GHz. At 3 GHz, the least energy is consumed. Notably, Figure 3d shows isolated unrealistic spikes, which exceed 1600 W, in the converted values of the last iteration. However, most measurements remain below 300 W, and do not exceed the TDP.

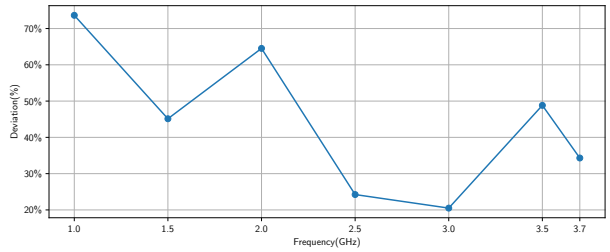
4.2.2 GROMACS. The deviation between the two energy consumption values for GROMACS is significantly higher than for STREAM, ranging between 45% and 140%, as can be seen in Figure 4a and Table 3. A trend similar to STREAM can not be recognized. The deviation starts at around 70% at 1 GHz and trends slightly downwards before reaching the minimum at 2.5 GHz with approximately 45%. After this point, the deviation increases sharply and reaches a maximum at 3.5 GHz with 140%, before decreasing again to around 85% for 3.7 GHz. The clock speed pattern in Figure 4b is similar to STREAM, with the effect of increasing the set frequency also disappearing above 2.5 GHz. While higher maximum frequencies of over 3 GHz are achieved, most remain around 2.75 GHz. The fluctuations again increase with increasing frequency. The total

Table 3: Data of the deviation between the Power(W) and Energy(J) measurements in table form.

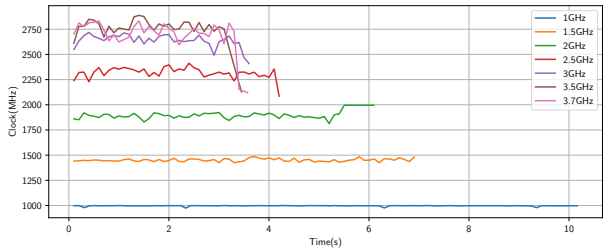
Application	1 GHz	1.5 GHz	2 GHz	2.5 GHz	3 GHz	3.5 GHz	3.7 GHz
STREAM	73.64 %	45.13 %	64.52 %	24.24 %	20.50 %	48.82 %	34.31 %
GROMACS	70.59 %	69.57 %	63.18 %	46.65 %	118.28 %	139.56 %	85.02 %
CIAO	1.15 %	1.41 %	1.44 %	1.09 %	1.08 %	1.03 %	0.91 %

Table 4: Data of the total energy consumption calculated using the Energy(J) measurements in table form.

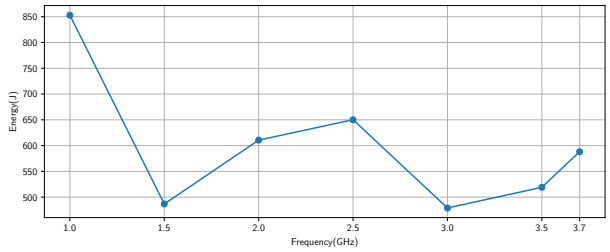
Application	1 GHz	1.5 GHz	2 GHz	2.5 GHz	3 GHz	3.5 GHz	3.7 GHz
STREAM	852.95 J	486.81 J	610.61 J	650.16 J	479.03 J	519.09 J	588.09 J
GROMACS	1020.87 J	1356.27 J	1056.54 J	1545.07 J	1450.75 J	1580.90 J	952.12 J
CIAO	1819.20 J	2004.24 J	2815.33 J	2250.61 J	2318.35 J	2331.71 J	2359.12 J



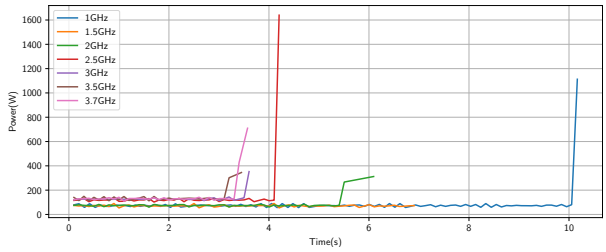
(a) Deviation between the Power(W) and converted Energy(J) measurements.



(b) Average frequency of all cores per iteration.



(c) Total energy consumption per frequency configuration calculated using the Energy(J) measurements.

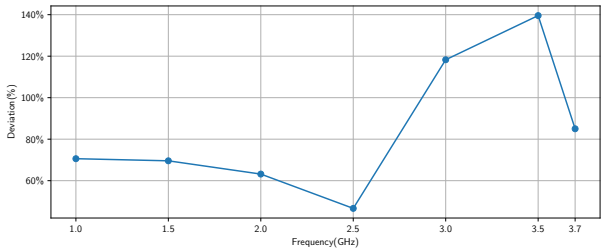


(d) Power draw per frequency configuration when converted from the Energy(J) measurement.

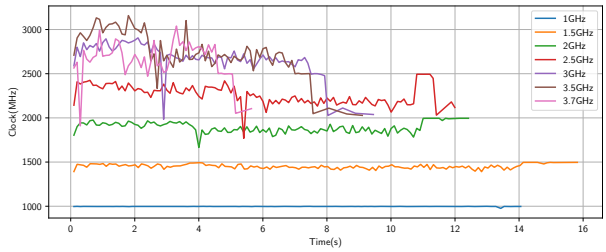
Figure 3: Power and energy measurements of STREAM with frequency scaling.

energy consumption behavior in Figure 4c is also different from STREAM. The energy consumption is lowest, around 1000 J, for 1 GHz, 2 GHz, and 3.7 GHz. It is higher, between 1350 J and 1600 J, for the other frequencies. When looking at the converted power values in Figure 4d, on which these results are based, it is noticeable that they show unrealistic outliers in the last few iterations, slightly skewing the total consumption values. This is similar to STREAM, but more pronounced. Furthermore, the TDP of 300 W is not reached but, excluding the outliers in the final iterations, a maximum of 200 W.

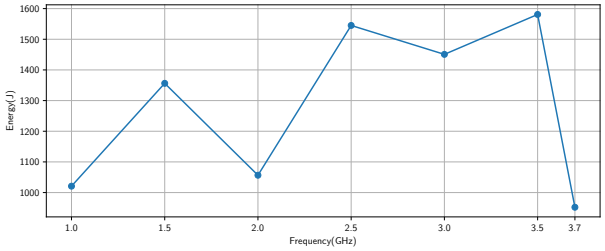
4.2.3 CIAO. The CIAO application behaves differently from both STREAM and GROMACS, as can be seen in Figure 5. The deviation between the two energy consumption measurements is in a low range of 0.9% to 1.45%. However, the frequencies shown in Figure 5b are not at the previously defined values but all range around 3 GHz, which is similar to the maximum observed in GROMACS. They are also more stable than in the other applications. But this is not the case for the total energy consumption in Figure 5c and Table 4, which displays differences of over 60%. We can observe a relationship between the execution time and total energy consumption. The lowest total consumption and also the shortest execution time is at 1 GHz. Meanwhile, the highest total consumption is at 2 GHz, which has the longest execution time and also reaches the highest frequencies for the longest time. Figure 5d shows that CIAO repeatedly spikes above the TDP to almost 350 W, unlike the other applications, which only exhibit such behavior in their last iterations.



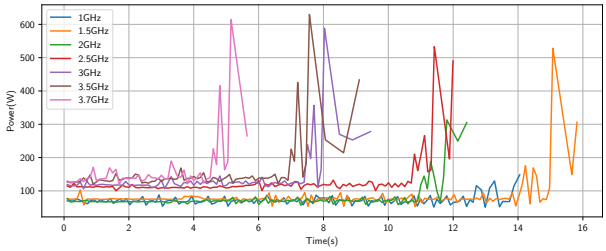
(a) Deviation between the Power(W) and converted Energy(J) measurements.



(b) Average frequency of all cores per iteration.

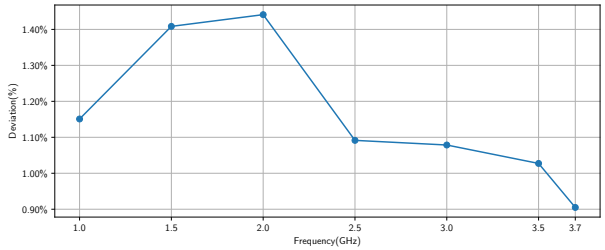


(c) Total energy consumption per frequency configuration calculated using the Energy(J) measurements.

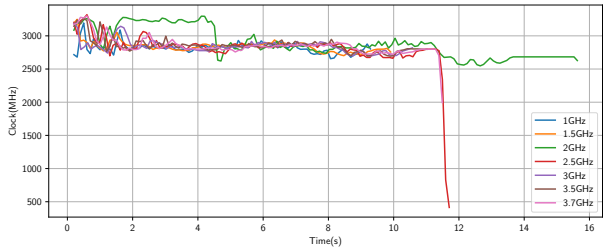


(d) Power draw per frequency configuration when converted from the Energy(J) measurement.

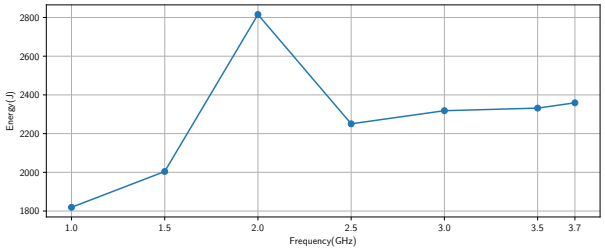
Figure 4: Power and energy measurements of GROMACS.



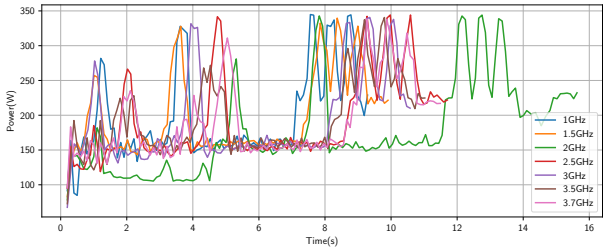
(a) Deviation between the Power(W) and converted Energy(J) measurements.



(b) Average frequency of all cores per iteration



(c) Total energy consumption per frequency configuration calculated using the Energy(J) measurements.



(d) Power draw per frequency configuration when converted from the Energy(J) measurement.

Figure 5: Power and energy measurements of CIAO.

5 DISCUSSION

In this section, we examine the results and process of our study in relation to our goals. We do this in two parts, first for the measurement results, and second for the energy profiling tool.

5.1 Results

5.1.1 STREAM. While not taking on the exact same values, our results of the measurement interval deviation on both systems are very similar to those reported by Kraljic et al. [18] for CPUs. This highlights the general applicability and transferability of the used methodology.

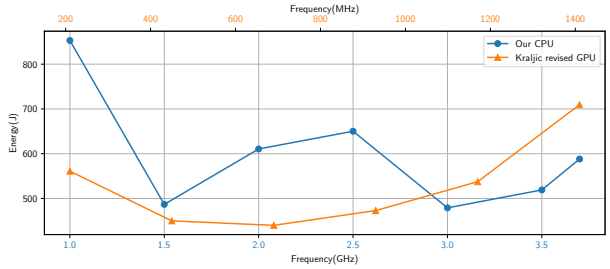


Figure 6: Comparison between the revised GPU results of Kraljic et al. [18] and our CPU results.

In the frequency scaling investigation, the total consumption of STREAM from Figure 3c has an unexpected course in which there is a local maximum in the middle-frequency range. This behavior is inverse when compared to the reference paper, which had the lowest consumption in that range. Our values can be explained by the continuously higher energy consumption of the middle frequencies during the runtime. We can recognize two distinct power bands in which each frequency setting approximately falls, 100 W and 150 W. Other differences in total consumption are due to the different runtimes caused by lower frequencies and, therefore, lower memory bandwidth utilization. Furthermore, it can be seen that STREAM has a constant profile in its power draw, fluctuating only slightly during the runtime, which shows that the benchmark has no particular phases that trigger a higher computational load and power consumption.

Our results are in line with previous research [4]. However, we noticed a discrepancy with the reference work in that their total consumption results are lower than ours by a factor of around 100, despite a similar TDP. We can attribute this to a minor problem in their calculations, which we can revise by using their provided graphs. For their NVIDIA node 1 running STREAM, we converted their values using the common $energy = power \cdot time$ formula and compiled the results in Figure 6. We can see that these are in line with our previous research while not contradicting their original conclusions.

5.1.2 GROMACS. The total consumption in Figure 4c does not display a consistent pattern. We can see that there are no specific phases during its execution that trigger higher utilization or consumption. Another similarity is that there are two distinct power bands, 75 W and 125 W watts, in which the power consumption falls. The latter is exceeded more frequently than in STREAM. A possible cause for the benchmark not reaching the TDP may be that it is not able to utilize the full computing power of the CPU, due to bandwidth limitations like those that STREAM measures. The fluctuations over the course of the runtime are also more pronounced compared to STREAM. Differences in total consumption are again attributable to runtime and average consumption.

While these results are higher than in previous research [28], they are still in a comparable range. It is difficult to judge how large the differences actually are because of the different datasets and measurement methods with different granularity. Further investigations might consider a wider range of configurations and datasets to find the influencing factors, which is especially important for GROMACS with its many unique use cases. Such data might also be collected by other research institutions for their use cases, based on a standardized measurement methodology such as ours, and gathered in a central repository.

5.1.3 CIAO. Unlike the other applications, CIAO exhibits distinct phases of energy consumption, which manifest in strong surges in power consumption. An explanation for these spikes remains an open and intriguing question, that requires an investigation of the proprietary source code. As there are no significant differences in the practically achieved frequencies, the differences in total consumption can only be attributed to the different runtimes. One standout is the 2 GHz setting, which has a significantly longer runtime. However, we see this as an isolated case which does not affect our investigation.

The very low deviation between the two power measurements in CIAO, see Figure 5a and Table 3, can be explained by its unique execution requirements. CIAO operates as an MPI program and has as many ranks as there are cores, even on a single node. This also changes the usage of LIKWID, where instead measuring the whole application, it is executed separately for each rank, i.e. for each core. We found in preliminary tests that executing an application with only one core causes LIKWID to not exhibit any unrealistic outliers. As a result, the deviation of CIAO is very small. However, this behavior of the energy profiler should be investigated further to find the underlying mechanisms causing it.

Examining the measurement data and collected CLI output revealed a significant difference between LIKWIDs measurement time and CIAO's reported runtime. This amounted to several seconds, while the total runtime did not even reach 30 seconds. The exact cause remains elusive, as the program's configuration and results are correct. In addition, the application seems to either explicitly ignore or remove the set frequencies. Further investigation in cooperation with the creators of the application is necessary to determine the causes and find energy-efficient settings.

5.2 Energy Profiler

Right settings are important for the energy profiling tools to get consistently correct results, and we learned several best practices that can be useful for further replication and for standardizing the energy characterization process.

During our testing, we learned that version 5.2 of LIKWID is currently best suited for our experiments. When we tried version 5.3 on CLAIX-23, it was not able to write timeline data to CSV files. This requires us to create additional log data processing to consolidate our results.

Regarding such timeline data when collected for MPI programs, it is important to consider the different execution conditions. Correct configurations require the tool's execution per MPI rank, leading to independent measurements. As such, the resulting data has to be synchronized and merged to obtain a complete picture. We also recommend using *likwid-perfctr* instead of *likwid-mpirun*, as it is more flexible and easier to work with.

As shown by our analysis, power, and energy data should be exclusively collected from the `Energy(J)` results. It still includes some spikes and deviations. However, they only have a minor influence on our findings. To further reduce their influence, we recommend

removing the first measurement iteration from the results since they consistently show implausible values.

6 CONCLUSION & FUTURE WORK

With this work we accomplished the goals we set out to achieve. We replicated the work of Kraljic et al. [18], and showed that it is transferable to different hardware and software environments. Furthermore, it is also applicable to CPUs and works for various applications, as evidenced by our measurement results, that expand the library of energy consumption investigations with three common HPC applications. Although we were able to transfer the methodology and achieve relatively comparable results, we still see clear differences. One aspect is the relationship between total energy consumption and frequency settings, which is quite unclear in our case, making it difficult to draw conclusions about the frequency setting for an optimal energy efficiency. Our current investigation of these results indicates that the variations are likely due to runtime differences and the practically achieved clock speed.

For future work, we want to continue to build on our results by providing a higher granularity on the performance measurements and more varieties in the methodology and energy profilers to test. Moreover, further in-depth investigations can be conducted to identify the underlying causes of the observed patterns, especially in the case of CIAO with its distinctive patterns. The findings from such investigations can reveal novel approaches to a creating an energy saving strategy tailored to an applications energy consumption pattern. We are aware that expanding the amount of energy investigations comes with the downside of increasingly scattered data over many works. Therefore, another step of our work is to create an open data repository for energy patterns, similar to recent initiatives in I/O tracing [24], to collect and combine energy pattern information from different data centers. Such a repository can serve many purposes, such as checking new hard- or software configurations with previous data, enabling comparisons between clusters, and aiding optimization efforts. It would also enable a comparison over time of how energy consumption changes with new hardware and optimization methods.

To illustrate the types and structure of data that might be stored in such a repository, we provide the data and scripts used to produce our results at https://doi.org/10.5281/zenodo.11086695. All of this, having more available data and understanding the causes and variables at play, will show new pathways to optimize energy efficiency. As a start for this work, we are performing our own initial characterization from two different systems and showing comparability to the existing works that can be reproduced and replicated by another interested party.

ACKNOWLEDGMENTS

This work is part of the EE-HPC project. EE-HPC is funded by BMBF Germany under the funding program Hoch- und Höchstleistungsrechnen für das digitale Zeitalter 2021-2024 – Forschung und Investitionen zum High-Performance Computing, Call Richtlinie zur Förderung von Verbundprojekten auf dem Gebiet des energieeffizienten High-Performance Computings (GreenHPC) with Grant No. 16ME0583K.

Computations were performed with computing resources granted by RWTH Aachen University under project rwth1496.

REFERENCES

[1] Mark James Abraham, Teemu Murtola, Roland Schulz, Szilárd Páll, Jeremy C. Smith, Berk Hess, and Erik Lindahl. 2015. GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers. *SoftwareX* 1-2 (2015), 19–25. https://doi.org/10.1016/j.softx.2015.06.001

[2] Advanced Micro Devices Inc. 2018. *ROCm System Management Interface (ROCm SMI) Library.* https://github.com/ROCm/rocm_smi_lib

[3] Advanced Micro Devices Inc. 2021. *ROC profiler library.* https://github.com/ROCm/rocprofiler

[4] Ghazanfar Ali, Mert Side, Sridutt Bhalachandra, Nicholas J. Wright, and Yong Chen. 2023. Performance-Aware Energy-Efficient GPU Frequency Selection Using DNN-Based Models. In *Proceedings of the 52nd International Conference on Parallel Processing (ICPP '23)*. Association for Computing Machinery, New York, NY, USA, 433–442. https://doi.org/10.1145/3605573.3605600

[5] Lukas Alt. 2023. *Performance metrics for access pattern-aware analysis of heterogeneous memory power consumption in HPC.* Masterarbeit. RWTH Aachen University, Aachen. https://doi.org/10.18154/RWTH-2023-10152 Veröffentlicht auf dem Publikationsserver der RWTH Aachen University; Masterarbeit, RWTH Aachen University, 2023.

[6] Shajulin Benedict. 2012. Energy-aware performance analysis methodologies for HPC architectures—An exploratory study. *Journal of Network and Computer Applications* 35, 6 (2012), 1709–1719. https://doi.org/10.1016/j.jnca.2012.08.003

[7] Enrico Calore, Alessandro Gabbana, Sebastiano Fabio Schifano, and Raffaele Tripiccione. 2020. ThunderX2 Performance and Energy-Efficiency for HPC Workloads. *Computation* 8, 1 (2020). https://doi.org/10.3390/computation8010020

[8] Stefano Corda, Bram Veenboer, and Emma Tolley. 2022. PMT: Power Measurement Toolkit. In *2022 IEEE/ACM International Workshop on HPC User Support Tools (HUST)*. 44–47. https://doi.org/10.1109/HUST56722.2022.00011

[9] Robert Curtis, Chris Peterson, David Moss, David Hardy, Rick Eiland, Tim Shedd, Eric Tunks, Hasnain Shabbir, and Todd Mottershead. 2023. *The Future of Server Cooling - Part 2: New IT hardware Features and Power Trends.* Retrieved March 22, 2024 from https://infohub.delltechnologies.com/p/the-future-of-server-cooling-part-2-new-it-hardware-features-and-power-trends-1/

[10] Olivier Desjardins, Guillaume Blanquart, Guillaume Balarac, and Heinz Pitsch. 2008. High order conservative finite difference scheme for variable density low Mach number turbulent flows. *Journal of Computational Physics* 227, 15 (2008), 7125–7159. https://doi.org/10.1016/j.jcp.2008.03.027

[11] Blake W. Ford and Ziliang Zong. 2021. PortAuthority: Integrating energy efficiency analysis into cross-platform development cycles via dynamic program analysis. *Sustainable Computing: Informatics and Systems* 30 (2021), 100530. https://doi.org/10.1016/j.suscom.2021.100530

[12] Daniel Hackenberg, Robert Schöne, Daniel Molka, Matthias S Müller, and Andreas Knüpfer. 2010. Quantifying power consumption variations of HPC systems using SPEC MPI benchmarks. *Computer Science-Research and Development* 25 (2010), 155–163. https://doi.org/10.1007/s00450-010-0118-0

[13] Thomas Ilsche. 2020. *Energy Measurements of High Performance Computing Systems: From Instrumentation to Analysis.* Ph. D. Dissertation. Technische Universität Dresden. https://nbn-resolving.org/urn:nbn:de:bsz:14-qucosa2-716000

[14] Intel Corporation. 2013. *Intel Xeon Processor E5-2697 v2.* Retrieved December 05, 2023 from https://ark.intel.com/content/www/us/en/ark/products/75283/intel-xeon-processor-e5-2697-v2-30m-cache-2-70-ghz.html

[15] Intel Corporation. 2023. *Intel Xeon CPU Platinum 8468 Processor.* Retrieved February 07, 2024 from https://www.intel.com/content/www/us/en/products/sku/231735/intel-xeon-platinum-8468-processor-105m-cache-2-10-ghz/specifications.html

[16] Y. Jiao, H. Lin, P. Balaji, and W. Feng. 2010. Power and Performance Characterization of Computational Kernels on the GPU. In *2010 IEEE/ACM Int'l Conference on Green Computing and Communications & Int'l Conference on Cyber, Physical and Social Computing*. 221–228. https://doi.org/10.1109/GreenCom-CPSCom.2010.143

[17] Hamidreza Khaleghzadeh. 2019. *Novel Data-Partitioning Algorithms for Performance and Energy Optimization of Data-Parallel Applications on Modern Heterogeneous HPC Platforms.* Ph. D. Dissertation. University College Dublin. https://hcl.ucd.ie/system/files/thesis-hamid.pdf

[18] Karlo Kraljic, Daniel Kerger, and Martin Schulz. 2022. Energy Efficient Frequency Scaling on GPUs in Heterogeneous HPC Systems. In *Architecture of Computing Systems (ARCS 2022)*. Springer International Publishing, Cham, 3–16. https://doi.org/10.1007/978-3-031-21867-5_1

[19] A. Kreuzer, E. Suarez, N. Eicker, and Th. Lippert (Eds.). 2021. *Porting applications to a Modular Supercomputer-Experiences from the DEEP-EST project.* Schriften des Forschungszentrums Jülich IAS Series, Vol. 48. Forschungszentrum Jülich, Jülich. https://juser.fz-juelich.de/record/905738

[20] Carsten Kutzner. [n. d.]. *A free GROMACS benchmark set.* Dept. of Theoretical and Computational Biophysics, Max Planck Institute for Multidisciplinary Sciences. Retrieved March 23, 2024 from https://www.mpinat.mpg.de/grubmueller/bench

[21] Radita Liem, Sebastian Oeste, Jay Lofstead, and Julian Kunkel. 2023. Mango-IO: I/O Metrics Consistency Analysis. In *2023 IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*. 18–24. https://doi.org/10.1109/CLUSTERWorkshops61457.2023.00013

[22] Matthias Maiterth, Gregory Koenig, Kevin Pedretti, Siddhartha Jana, Natalie Bates, Andrea Borghesi, Dave Montoya, Andrea Bartolini, and Milos Puzovic. 2018. Energy and Power Aware Job Scheduling and Resource Management: Global Survey — Initial Analysis. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 685–693. https://doi.org/10.1109/IPDPSW.2018.00111

[23] John D McCalpin. 1995. Memory bandwidth and machine balance in current high performance computers. *IEEE computer society technical committee on computer architecture (TCCA) newsletter* 2, 19-25 (1995).

[24] Nafiseh Moti, André Brinkmann, Marc-André Vef, Philippe Deniel, Jesus Carretero, Philip Carns, Jean-Thomas Acquaviva, and Reza Salkhordeh. 2023. The I/O Trace Initiative: Building a Collaborative I/O Archive to Advance HPC. In *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. 1216–1222. https://doi.org/10.1145/3624062.3624192

[25] Ludovico Nista, Christoph Schumann, Temistocle Grenga, Antonio Attili, and Heinz Pitsch. 2023. Investigation of the generalization capability of a generative adversarial network for large eddy simulation of turbulent premixed reacting flows. *Proceedings of the Combustion Institute* 39, 4 (2023), 5279–5288. https://doi.org/10.1016/j.proci.2022.07.244

[26] NVIDIA Corporation. 2016. *nvidia-smi documentation.* Retrieved December 07, 2023 from https://developer.download.nvidia.com/compute/DCGM/docs/nvidia-smi-367.38.pdf

[27] NVIDIA Corporation. 2024. *NVML API Reference Guide: GPU Deployment and Management Documentation.* Retrieved May 2, 2024 from https://docs.nvidia.com/deploy/nvml-api/index.html

[28] Armen Poghosyan, Hrachya Astsatryan, Wahi Narsisian, and Yevgeni Mamasakhlisov. 2017. On the Performance and Energy Consumption of Molecular Dynamics Applications for Heterogeneous CPU-GPU Platforms Based on Gromacs. *Cybernetics and Information Technologies* 17, 5 (2017), 68–80. https://doi.org/10.1515/cait-2017-0056

[29] Thomas Röhl, Jan Treibig, Georg Hager, and Gerhard Wellein. 2014. Overhead Analysis of Performance Counter Measurements. In *2014 43rd International Conference on Parallel Processing Workshops*. 176–185. https://doi.org/10.1109/ICPPW.2014.34

[30] Shuaiwen Song, Rong Ge, Xizhou Feng, and Kirk W. Cameron. 2009. Energy Profiling and Analysis of the HPC Challenge Benchmarks. *The International Journal of High Performance Computing Applications* 23, 3 (2009), 265–276. https://doi.org/10.1177/1094342009106193

[31] top500.org. [n. d.]. *Performance Development.* Retrieved March 22, 2024 from https://www.top500.org/statistics/perfdevel/

[32] Wendy Torell. 2019. *It's not just about chip density: Considering liquid cooling for your data center.* Retrieved March 22, 2024 from https://www.datacenterdynamics.com/en/opinions/its-not-just-about-chip-density-considering-liquid-cooling-for-your-data-center/

[33] Wallace Witkowski. 2022. *'Moore's Law's dead,' Nvidia CEO Jensen Huang says in justifying gaming-card price hike .* Retrieved December 05, 2023 from https://www.marketwatch.com/story/moores-laws-dead-nvidia-ceo-jensen-says-in-justifying-gaming-card-price-hike-11663798618

[34] Andrew J. Younge, Ryan E. Grant, James H. Laros, Michael Levenhagen, Stephen L. Olivier, Kevin Pedretti, and Lee Ward. 2019. Small scale to extreme: Methods for characterizing energy efficiency in supercomputing applications. *Sustainable Computing: Informatics and Systems* 21 (2019), 90–102. https://doi.org/10.1016/j.suscom.2018.11.005